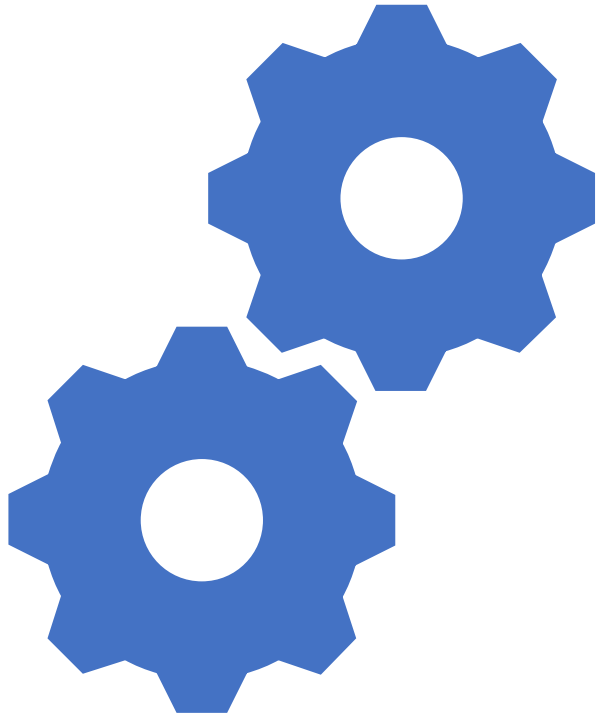


Software Re-Engineering

A thick, hand-drawn style orange line that spans approximately two-thirds of the width of the slide, positioned directly below the main title.

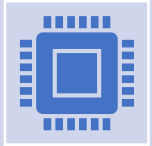
Week 2



Reverse Engineering

muhammadfawad@cuilahore.edu.pk

Reverse Engineering



Reverse engineering is the process of analyzing a system (software, hardware, or both) to extract design and implementation information.



This is often done when source code or design documents are unavailable.



It can involve both static and dynamic techniques to achieve various goals like understanding, improving, or replicating a system.

Goals of Reverse Engineering

Analyze	Analyze security vulnerabilities.
Recover	Recover lost or unavailable design documents.
Enable	Enable system maintenance and modernization.
Ensure	Ensure interoperability and integration with other systems.

Techniques for Reverse Engineering



Static Techniques



Dynamic Techniques



Hybrid Techniques

Static Techniques

Static Technique

The process of analyzing a program or system without executing it.

This involves examining the structure, code, or binary of the software to understand its design and functionality.

Static Analysis Techniques



Decompilation



Disassembly



Model
Extraction



Binary
Inspection



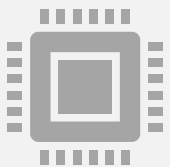
Signature-
based Analysis



Decompilation



It refers to converting binary code back into higher-level programming languages, like C++, Java or Python. While it's difficult to recover the exact original code, decompilers generate pseudocode that extracts the original logic, making it easier to understand.



Decompilation is particularly useful when you want to retrieve the logic or algorithms used in the program for modification, debugging, or analysis purposes. It is also valuable in identifying vulnerabilities or understanding malware behaviors.

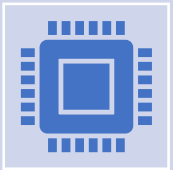
Ghidra: Ghidra features a built-in decompiler for reverse engineering various architectures.

IDA Pro: It also includes an advanced decompiler that generates high-level pseudocode, making it easier to reverse engineer complex software.

Disassembly



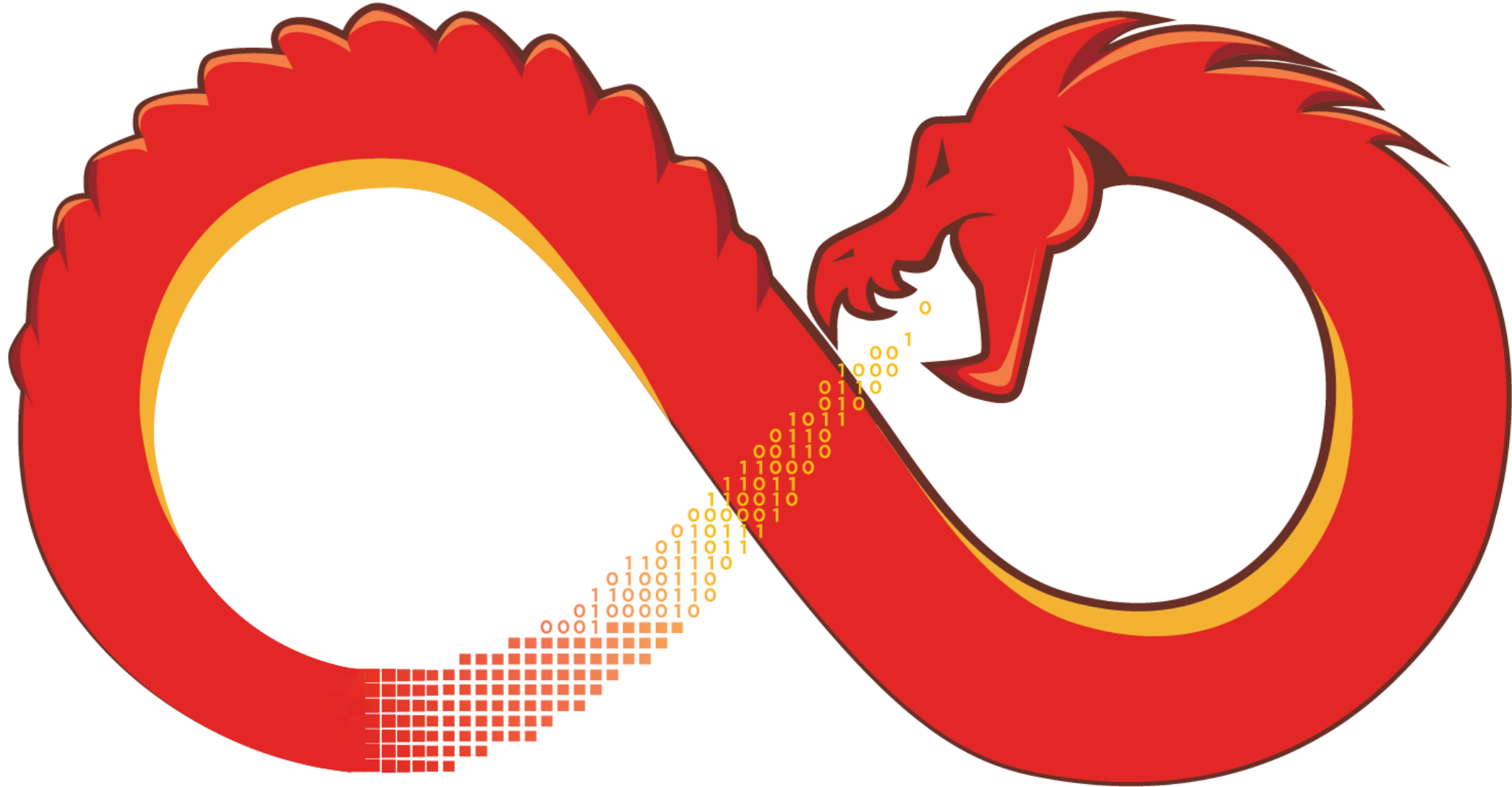
Disassembly refers to converting machine code or bytecode into assembly code. It allows to view the low-level instructions used by the computer's CPU.



Disassembly is used to analyze how a binary works by identifying the functions, loops, and system calls made by the program. It's commonly used for malware analysis, and vulnerability discovery.

Ghidra: It includes a powerful disassembler that supports various processor architectures, and it provides a user-friendly interface to view code flow, make changes, and run scripts.

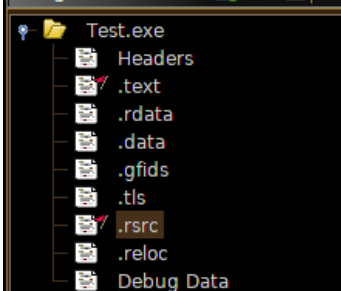
IDA Pro: It can disassemble and analyze complex binaries. It has a built-in decompiler to convert machine code back into more human-readable C-like pseudocode.



GHIDRA

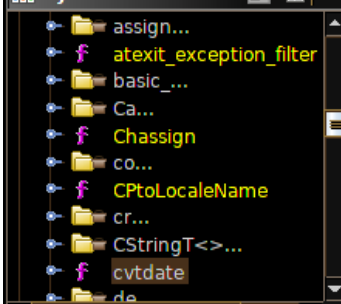


Program Trees



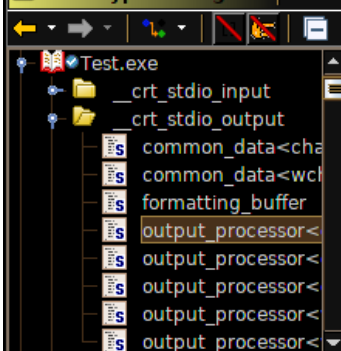
Program Tree

Symbol Tree



Filter:

Data Type Manager

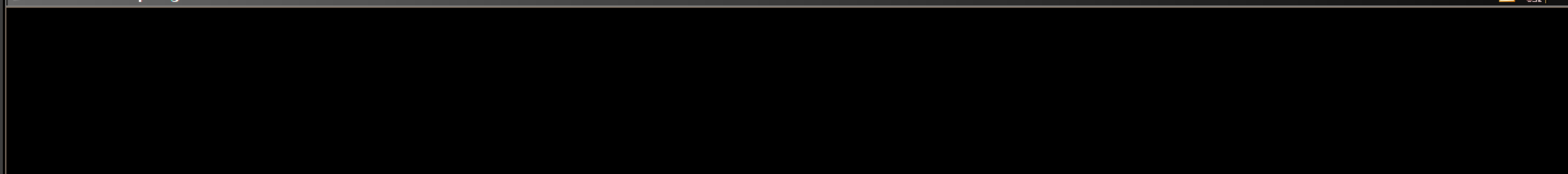


Filter:

Decompile: cvtdate - (Test.exe)

```
31     iVar7 = *(int *)(&DAT_0072fa6c + iVar6);
32 }
33 else {
34     iVar7 = *(int *)(&DAT_0072faa0 + iVar6);
35 }
36 iVar3 = __crt_time_elapsed_leap_years<int>(param_3);
37 iVar5 = (iVar3 + iVar7 + 1 + param_3 * 0x16d + -0x63ca) % 7;
38 iVar3 = param_5 * 7 - iVar5;
39 if (iVar5 <= param_6) {
40     iVar3 = iVar3 + -7;
41 }
42 iVar7 = iVar7 + 1 + iVar3 + param_6;
43 if (uStack8._3_1_ == 0) {
44     iVar6 = (&DAT_0072fa70)[param_4];
45 }
46 else {
47     iVar6 = *(int *)(&DAT_0072faa4 + iVar6);
48 }
49 if ((param_5 == 5) && (iVar6 < iVar7)) {
50     iVar7 = iVar7 + -7;
51 }
52 }
53 else {
54     bVar2 = __crt_time_is_leap_year<int>(param_3);
55     if (bVar2 == false) {
56         iVar7 = *(int *)(&DAT_0072fa6c + param_4 * 4);
57     }
58     else {
59         iVar7 = *(int *)(&DAT_0072faa0 + param_4 * 4);
60     }
61     iVar7 = iVar7 + param_7;
62 }
63 param_11 = ((param_8 * 0x3c + param_9) * 0x3c + param_10) * 1000 + param_11;
64 if (param_1 == 0) {
65     _DAT_00810b40 = param_3;
66     _DAT_00810b44 = iVar7;
67     _DAT_00810b48 = param_11;
68 }
69 }
70 else {
```

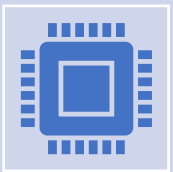
Console - Scripting



Model Extraction



It refers to the process of converting low-level code, such as binaries or source code, into high-level representations like UML diagrams, flowcharts, or data flow diagrams.



This technique is used to represent a system's architecture or behavior visually, making it easier for developers or analysts to understand the overall structure of the software.

Enterprise Architect: A comprehensive UML modeling tool that can reverse-engineer code into UML diagrams for various programming languages.

Eclipse MoDisco: A tool which is often used to extract architectural models from existing systems and generate UML diagrams from source code.

WSDDemo (2014) - Enterprise Architect

FILE EDIT VIEW PROJECT PACKAGE DIAGRAM ELEMENT TOOLS ANALYZER EXTENSIONS WINDOW HELP

Current Package Main Code Model VSLink

Find Package

x86 - Debug

Project Browser

ArchiMate View

Zachman Framework

TOGAF

Formal Requirements Model

Use Case Model

Traceability Model

Business Processes Model

Web Services Model

Code Model

Code Model

OurOnlineBookstore

PHP

VSLink

VSLink

Sparx_Domains

Persistence Model

Interface Design

Components

AltImages

Deployment

Project Browser Resources

Execution Analyzer

Main.Business Processes Model.Application Process

Main.Business Processes Model.SIMULATION

Main.Code Model.VSLink

x86 - Debug

x86 - Release

Main.Index.EA11.Demo.StateMachine Generation.DeepHistory

Main.Web Services Model.C# Model.Example..NET.Source

Main.Web Services Model.C# Model.Example..NET.Workbench

Main.Web Services Model.C++ Code Model.C++/Native.Source

Start Page

Functional Requirements

VSLink

Diagram

Sparx_Domains::Program

Sparx_Domains::Licenses

Sparx_Domains::cQuery

Sparx_Domains::gph_transactionsperday

Custom

Package

Requirement

Issue

Change

Screen

UI Control

Test Case

Entity

Custom Relationships

Common

Artifacts

Artifact

Document

Checklist

User Story

Working Set

Standard Chart

Time Series Chart

Model View

Report Specification

Matrix Specification

Executable StateMachine

cQuery.cs

C:\Users\toreilly\Documents\Visual Studio 2010\Projects\Sparx Domains\Sparx Domains\cQuery.cs

cQuery

1 using System;

2 using System.Collections.Generic;

3 using System.Collections;

4 using System.Linq;

5 using System.Text;

6 using System.Data.Odbc;

7 using System.Windows.Forms;

8 using System.Reflection;

9 using System.Data;

10

11 namespace Sparx_Domains

12 {

13 public class cQuery

14 {

15 public static cQuery INSTANCE = new cQuery();

16 public Hashtable h_CodeToDesc = new Hashtable();

17 public Hashtable h_DescToCode = new Hashtable();

18 public Hashtable h_VAT = new Hashtable();

19

20

21 public OdbcConnection DbConnection;

22 string conString;

23

24 private cQuery()

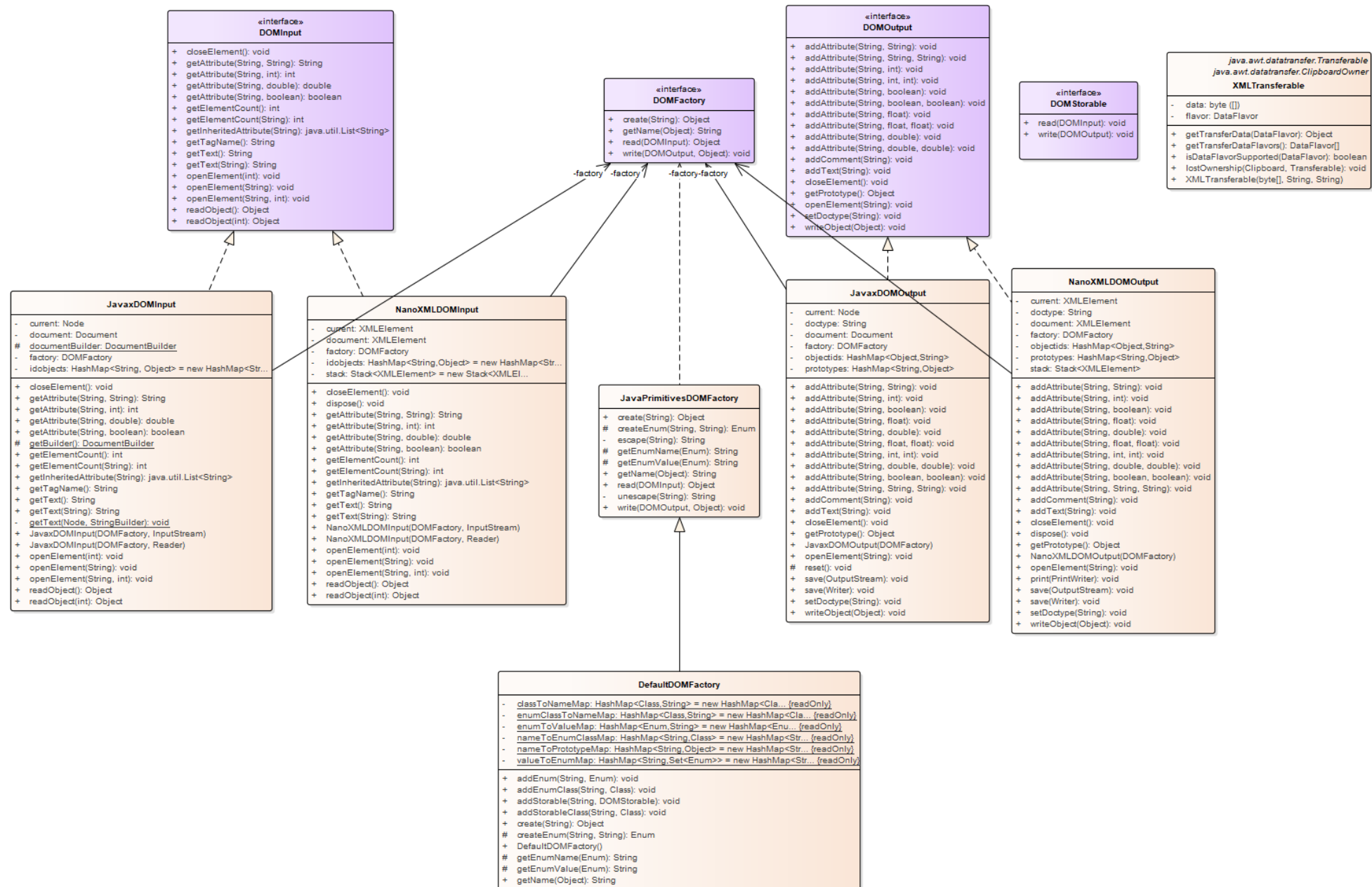
25 {

Debug

Execution Analyzer

Custom Diagram:VSLink: created: 11/12/2014 3:07:12 PM modified: 12/12/2014 5:01:44 PM 100% 783 x 1126

CAP NUM SCRL CLOUD



Business Process Modeling Frameworks

- **Archimate View**

- It provides a uniform way to describe the construction and operation of business processes, organizational structures, information flows, IT systems, and technical infrastructures.

- **Business Layer**

- Focused on how an organization operates, encompassing its processes, people, and products.

- **Application Layer**

- Describes the software systems and applications that support the business processes.

- **Technology Layer**

- Focuses on the physical and technological infrastructure that supports the application layer

Business Process Modeling Frameworks

- **Zachman Framework**

- It provides a structured way of viewing and defining an enterprise. It was developed by John Zachman in 1987 and is often referred to as a taxonomy for enterprise architecture.
- The Framework is structured as a two-dimensional grid. It consists of six columns representing different perspectives and six rows representing different aspects of an enterprise.

Business Process Modeling Frameworks

Perspective / Aspect	What (Data)	How (Function)	Where (Network)	Who (People)	When (Time)	Why (Motivation)
Planner	Contextual Data	Contextual Function	Contextual Network	Contextual People	Contextual Time	Contextual Motivation
Owner	Business Data	Business Function	Business Network	Business People	Business Time	Business Motivation
Designer	System Data	System Function	System Network	System People	System Time	System Motivation
Builder	Technology Data	Technology Function	Technology Network	Technology People	Technology Time	Technology Motivation
Subcontractor	Component Data	Component Function	Component Network	Component People	Component Time	Component Motivation
Enterprise	Detailed Data	Detailed Function	Detailed Network	Detailed People	Detailed Time	Detailed Motivation

Business Process Modeling Frameworks

- **TOGAF (The Open Group Architecture Framework)**

- It is used by organizations to design, plan, implement, and govern enterprise architecture. It provides a structured approach for designing, managing, and improving an organization's information technology.
 1. **Preliminary Phase:** Establishing the architecture framework.
 2. **Architecture Vision:** Developing a high-level vision of the project.
 3. **Business Architecture:** Designing the business process architecture.
 4. **Information Systems Architecture:**
 - **Data Architecture:** Structure of an organization's data assets.
 - **Application Architecture:** Software applications supporting business processes.
 5. **Technology Architecture:** Infrastructure supporting the business processes.
 6. **Opportunities and Solutions:** Identifying candidate solutions for implementation.
 7. **Migration Planning:** Creating a roadmap for implementation.
 8. **Implementation Governance:** Ensuring compliance with the architecture.
 9. **Architecture Change Management:** Managing changes to the architecture.

Binary Inspection

It refers to the examination and analysis of a compiled program's binary code without executing it.

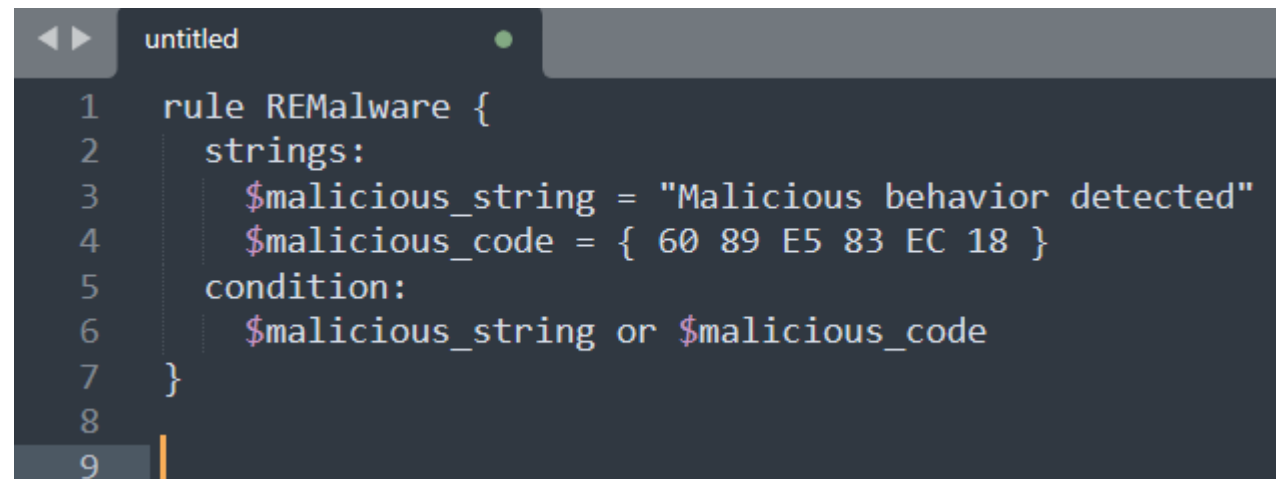


The binary file (e.g., .exe, .dll, or other compiled formats) is inspected to understand the underlying structure, functionality, and behavior of the software.

Disassembly	Decompilation	Code Patterns	Symbol Tables and Metadata	String Analysis	Hexadecimal Representation	File Structure	Control Flow Analysis	Data Flow Analysis
-------------	---------------	---------------	----------------------------	-----------------	----------------------------	----------------	-----------------------	--------------------

Signature-based Analysis

- A method where specific known patterns or “signatures” in a program's binary code are compared to a database of predefined signatures.
- This technique is commonly used to detect known malware, identify functions, or recognize particular code behaviors by looking for matching byte sequences, API calls, or other distinct code characteristics.
 - YARA
 - Antivirus Engines
 - Ghidra
 - IDA Pro

A screenshot of a code editor window titled 'untitled'. The editor displays a YARA rule named 'REMalware'. The rule is structured with a 'strings' section containing two variables: '\$malicious_string' set to the text 'Malicious behavior detected' and '\$malicious_code' set to a byte sequence { 60 89 E5 83 EC 18 }. The 'condition' section follows, stating '\$malicious_string or \$malicious_code'. The code is syntax-highlighted with blue for keywords, green for strings, and red for the byte sequence. Line numbers 1 through 9 are visible on the left side of the editor.

```
1 rule REMalware {  
2     strings:  
3         $malicious_string = "Malicious behavior detected"  
4         $malicious_code = { 60 89 E5 83 EC 18 }  
5     condition:  
6         $malicious_string or $malicious_code  
7 }  
8  
9
```

Files

☒ APT_APT1.yar☐ APT_APT10.yar☐ APT_APT15.yar☐ APT_APT17.yar

Rules

APT1_WEBC2_AUSOV

APT1_WARP

APT1_TARSIP_ECLIPSE

APT1_TARSIP_MOON

APT1_payloads

APT1_RARSilent_EXE_PDF

APT1_aspnetreport

APT1_Revird_svc

APT1_dbg_mess

APT1_known_malicious_RARSilent

```
1178
1179 meta:
1180   author = "AlienVault Labs"
1181   info = "CommentCrew-threat-apt1"
1182
1183 strings:
1184   $winrar1 = "WINRAR.SFX" wide ascii
1185   $str2 = "Steup=" wide ascii
1186
1187 condition:
1188   all of them
1189 }
1190
1191 rule APT1_aspnetreport
1192 {
1193
1194   meta:
1195     author = "AlienVault Labs"
1196     info = "CommentCrew-threat-apt1"
1197
1198   strings:
1199     $url = "aspnet_client/report.asp" wide ascii
1200     $param = "name=%s&Gender=%c&Random=%04d&SessionKey=%s" wide ascii
1201     $pay1 = "rusinfo.exe" wide ascii
1202     $pay2 = "cmd.exe" wide ascii
1203     $pay3 = "AdobeUpdater.exe" wide ascii
1204     $pay4 = "buildout.exe" wide ascii
1205     $pay5 = "DefWatch.exe" wide ascii
1206     $pay6 = "d.exe" wide ascii
1207     $pay7 = "em.exe" wide ascii
1208     $pay8 = "IMSCMig.exe" wide ascii
1209     $pay9 = "localfile.exe" wide ascii
1210     $pay10 = "md.exe" wide ascii
1211     $pay11 = "mdm.exe" wide ascii
1212     $pay12 = "mimikatz.exe" wide ascii
1213     $pay13 = "msdev.exe" wide ascii
1214     $pay14 = "ntoskml.exe" wide ascii
1215     $pay15 = "r.exe" wide ascii
```

File

Line

Message

Dynamic Techniques

A thick, hand-drawn style orange line that underlines the text "Dynamic Techniques". It starts under the 'D' and ends under the 's', following the width of the text.

Dynamic Technique

- Dynamic analysis involves executing the software and monitoring its behavior in real-time.
- This approach helps to understand how the software interacts with the operating system, hardware, network, and other components under actual working conditions.

Dynamic Analysis Techniques



Tracing



Profiling



System
Monitoring



Debugging



Binary
Instrumentation



Tracing

- A technique that involves monitoring the execution flow of a program to record its behavior during runtime.
- It focuses on capturing critical runtime information, such as system calls, function invocations, memory access, and interactions between different components of the software or system.

Profiling

- A technique used to measure the performance of a program while it runs.
- It helps to understand how much time the program spends on different tasks, how much memory it uses, and how efficiently it uses system resources like the CPU and network.
 - A **bottleneck** is a part of the program that slows everything down.
 - Profiling helps you find these bottlenecks by showing which parts of the program take the longest to run or use the most resources.
 - CPU Profiling
 - Memory Profiling
 - I/O Profiling
 - Functional Level Profiling

System Monitoring

- Observing how a program interacts with the entire operating system and external resources in real time.
- It's mainly used to track external actions a program takes, such as reading/writing files, making network connections, or using system resources like memory and CPU.
- Tools
 - Process Monitor
 - Wireshark
 - Sysdig

Tracing vs Profiling vs System Monitoring

Aspect	Tracing	Profiling	System Monitoring
Purpose	Captures detailed, step-by-step execution of code	Measures performance (e.g., CPU, memory usage)	Tracks overall system health and resource usage
Scope	Focuses on the sequence of events or function calls	Aggregates resource usage (CPU, memory, I/O)	Broad view covering system-wide metrics (CPU, memory, disk, network traffic)
Output	Logs of events/method calls with timestamps	Statistical performance data per function/thread	Dashboard of metrics, logs, graphs, alerts
Granularity	Function-level data, event-level data	Aggregated, high-level/low-level performance metrics	System-wide resource consumption and performance trends
Use Case	Debugging, tracking flow of execution, identifying bugs	Performance optimization, finding bottlenecks	System health monitoring, detecting abnormal behavior
Examples	Logs method calls in an application's lifecycle	Shows which methods consume the most CPU or memory	Alerts when CPU usage spikes or memory is nearly full
Focus	Specific functions or processes	Code-level resource usage	Entire system performance
Tools	Android Systrace, Event Tracing for Windows (ETW)	Android Studio Profiler, VisualVM	System Activity Monitor, Nagios, Prometheus



Debugging

It involves running software in a controlled environment to step through its execution in real time, allowing to observe its behavior, monitor the state of variables, registers, and memory, and identify the flow of execution.

It helps to understand how software operates, detect vulnerabilities, and uncover hidden functionalities or obfuscated code.

Debugging



Breakpoints

Watchpoints

Step Execution

Call Stack Inspection

Register and Memory Inspection

Debugging

- **Breakpoints**

- Think of a breakpoint as a stop sign in your code. When the program reaches this point, it stops running.
- This allows you to pause the program, take a close look at what's happening.
- If you want to see what happens before a function executes, you set a breakpoint there. When the program stops, you can inspect its current state.

- **Watchpoints**

- It is like a break point, but instead of stopping at a specific part of the code, it stops when a certain piece of data changes.
- If you're tracking when a variable gets modified, you can set a watchpoint and the debugger will stop whenever that variable changes.

Debugging

- **Step Execution**

- Let you move through the program one step at a time. There are different ways to do this:
 - **Step into:** Go inside a function and see what happens inside.
 - **Step over:** Skip over the function but let it run normally.
 - **Step out:** Leave the current function and return to where it was called.
- It helps you understand what each part of the code does by slowly executing one line at a time.
- If a bug happens inside a function, you can “**Step into**” that function to see exactly what goes wrong.

Debugging

- **Call Stack Inspection**

- A history of all the functions that the program has called to reach the current point.
- Shows the path the program has taken, which helps you understand the sequence of function calls.
- If an error happens deep in the program, the call stack can show you which functions were called and in what order.

- **Register and Memory Inspection**

- **Registers** are small, fast storage locations in the CPU.
- **Memory** is where the program stores data while it runs.
- By inspecting memory and registers, you can see what data the program is handling and how it's being manipulated.

Binary Instrumentation

- A technique that allows you to modify a program's binary code while it is running.
- This helps in observing the program's behavior, analyze how it works, and identify vulnerabilities or bugs without having the original source code.
- Tools
 - Intel PIN
 - DynamoRIO
 - Valgrind

Questions?